

```

class NChannelTimeGates(nn.Module):
    def __init__(self, N=128, window_size=10):
        super().__init__()
        self.N = N # Число каналов
        self.window = window_size #
Временное окно (мс)

        # N временных детекторов (по
одному на канал)
        self.detectors = nn.ModuleList([
            nn.Sequential(
                nn.Linear(3, 16), # Триарный вход
{-1,0,+1}
                nn.ReLU(),
                nn.Linear(16, 1), # Бинарный
ВЫХОД {0,1}
                nn.Sigmoid()
            ) for _ in range(N)
        ])

    def forward(self, spike_stream):
        """

```

spike_stream: [batch, time_steps, 3] -
триарный поток спайков
////

```
batch, time, _ = spike_stream.shape  
gated_vectors = []
```

```
for t in range(time - self.window + 1):  
    # Временное окно [t:t+window]  
    window = spike_stream[:,  
t:t+self.window, :]
```

```
    # N параллельных детекторов  
    channel_activations = []  
    for i, detector in  
enumerate(self.detectors):  
        # Агрегация по времени в канале i  
        channel_signal = window[:, :, i % 3] #  
Канал i  
        activation =  
detector(channel_signal.mean(dim=1))  
  
channel_activations.append(activation)
```

```
# N-мерный синхронизированный  
вектор  
    xin = torch.stack(channel_activations,  
dim=1) # [batch, N]  
    gated_vectors.append(xin)  
  
    return torch.stack(gated_vectors, dim=1)  
# [batch, time, N]
```